

Mikhail Razakov

Compiler / Runtime Engineer

I develop UniversalToolchain/Wist2, a modular .NET compiler/runtime framework with BytecodeAIR, a reference interpreter and a Cil/DynamicMethod backend. I implemented an experimental verifier-gated AIR → SSA → AIR route: lowering/emission, constant folding, SCCP-lite, branch folding, unreachable cleanup and DCE. A separate PS-form Analyzer demonstrates conservative program analysis and exact-oracle testing.

- Moscow / remote
- HSE University student
- Available from September 2026 - up to 20 hours/week
- MCST Internship - July-August 2026

misha13022008@gmail.com · t.me/Micody · github.com/Misha1302 · LinkedIn

AIR → SSA → AIR lowering/emission and optimization passes under structural verifiers	1,325 tests · 75 .NET projects verified baseline dated July 14, 2026; 0 warnings and 0 errors	1st place · 104/104 conservative memory-dependence analyzer	
2024 — present creator of UniversalToolchain/Wist2	2 execution backends reference interpreter and Cil/DynamicMethod	IR → optimization → runtime explicit boundaries and capability contracts	x86 / x86-64 Assembly NASM/FASM · ABI · x87 · compiler output

01 · EXPERIENCE

Compilers, program analysis and semantic validation.

2024 — PRESENT	UniversalToolchain / Wist2 — Creator and Primary Developer Compiler/runtime framework · .NET · public alpha - Designed Source → AST → Bytecode → AIR → Backend → Execution, a reference interpreter, Cil/DynamicMethod backend and module composition. - Implemented an experimental opt-in AIR → SSA → AIR route: lowering/emission, constant folding, SCCP-lite, branch folding, unreachable cleanup and DCE. - SSA is verified after lowering and every pass; emitted AIR is verified before the backend; interpreter/CL parity covers supported semantics. - Verified baseline dated July 14, 2026: 75 .NET projects building, 1,325 passing tests, 0 warnings and 0 errors.
JULY-AUGUST 2026	MCST — Compiler Engineering Intern LLVM track · C++23 · remote - Developing graph-analysis components over a shared data model with explicit input, error and result contracts. - Maintaining CMake, warnings-as-errors, ASan/UBSan, clang-tidy, explicit include dependencies and reproducible I/O tests.
2021 — PRESENT	Programming Tutor and Mentor C/C++ · C# · Python · NASM · algorithms Taught about 50 students; I diagnose gaps, explain execution models and review code.

02 · PROJECTS

Compiler/runtime, program analysis and low-level backend experiments.

COMPILER/RUNTIME FRAMEWORK · .NET

UniversalToolchain / Wist2

A public modular framework for embedded DSLs with multiple IR layers and two execution backends.

GitHub [➤](#) Documentation [➤](#)

PROGRAM ANALYSIS · C17

PS-form Memory Dependence Analyzer

A conservative analyzer for memory dependencies across independent loop iterations.

- Ranked 1st out of 49; the only 5.0/5.0 and 104/104 result.
- Normalization, range/residue/GCD filters, exact affine analysis and bounded search.
- Exact oracle, randomized/metamorphic tests and counterexample retention.

GitHub [➤](#)

C17Program analysis104/104

ASSEMBLY AND BACKEND · NASM / RUST · IA-32 / X86-64

x86 Assembly, Codegen & Register Allocation

Hand-written NASM for IA-32 and an x86-64 code-generation lab, spanning ABI and stack mechanics through liveness, register allocation and native-code validation.

- NASM IA-32: cdecl, stack frames, libc calls, structures and addressing, x87; buildable examples and a practical course.
- x86-64: SSA-like IR, live intervals, interference graph, linear scan and lowering/emission through iced-x86.
- Disassembly and compiler-output analysis; differential tests, spill/code-size metrics and isolated execution.

Codegen lab [➤](#) NASM course [➤](#)

NASM/FASMIA-32 / x86-64ABI / x87

ALGORITHM LIBRARY · C++23

AdvancedAlgorithms

Broader C++ and algorithmic foundation: 12 reusable modules with explicit contracts.

- Graphs, trees, strings and data structures: centroid, HLD, LCA, flow, SCC, Aho-Corasick and more.
- Differential tests, structural invariants, invalid-input checks and ASan/UBSan.
- Large-input checks for stack safety and accidental O(n) behaviour.

GitHub [➤](#)

C++23AlgorithmsTesting

03 · EXPERTISE

Compiler pipelines, program analysis and semantic validation.

Architecture and IR	Lexing and parsing, AST, Bytecode and AIR; a reference interpreter and Cil/DynamicMethod code generation; modular composition and explicit capability contracts.
Analysis and optimization	CFG and SSA; AIR-to-SSA lowering and SSA-to-AIR emission; SCCP, constant and branch folding, unreachable-block elimination, dead pure-instruction elimination, liveness and memory-dependence analysis.
Correctness	Structural IR verification before and after transformations, interpreter/backend equivalence checks, exact reference models, differential and metamorphic tests, and negative cases.
Assembly and ABI	NASM/FASM, IA-32 and x86-64; cdecl and System V AMD64 ABI; stack frames, libc interop, structures and addressing, x87; GDB/objdump and disassembly analysis.
Languages and toolchain	C#/.NET, C++23, C17 and Python; CMake, GitHub Actions, ASan/UBSan and clang-tidy.

04 · RECOGNITION

Olympiad and project achievements.

2025–2026	Programming olympiads Prize-winner of the HSE “Vysshaya Proba” Olympiad in competitive and industrial programming; prize-winner of the regional Russian Informatics Olympiad. Vysshaya Proba ➤
2025–2026	Compiler projects Two highest-tier diplomas in the MEPhI Junior competition and a highest-tier diploma in the Baltic Science and Engineering Competition for compiler/runtime projects. MEPhI Junior 2025 ➤ MEPhI Junior 2026 ➤ Baltic competition ➤
2025–2026	HSE “Vysshiy Pilotazh” · Computer Science Winner of the Computer Science track. Results ➤

05 · EDUCATION

Education and availability.

HSE University student.

STUDENT	HSE University Moscow HSE University student.
FROM SEP 2026	Internship or part-time role Moscow / remote Looking for work in compiler/runtime engineering, program analysis, compiler testing, or language tooling.